University Mohammed VI Polytechnic

College of Computing

Lab 5: Normalization

BCNF Analysis, Decomposition and Dependency Preservation

Data Management Course

UM6P College of Computing

Professor: Karima Echihabi **Program:** Computer Engineering **Session:** Fall 2026

Team Information

Member 1	Malak Nhami
Member 2	Marwa Mazouz
Member 3	Mohamed Moussaid
Member 4	Sara Lamnaiy
Repository Link	`https://github.com/malak-3108/project`

1 Relations of the Lab 4 Solution that are not in BCNF

Answer: none. Every relation of the Lab 4 solution satisfies BCNF, because no relation contains a non-trivial functional dependency whose determinant is not a superkey.

Definition. A relation schema R is in BCNF if for every non-trivial FD $X \to Y$ that holds over R, X is a superkey of R. In the Lab 4 solution, every non-trivial FD has the primary key (simple or composite) as its determinant.

Relation	Non-trivial FD	Superkey?
Person, Student, Employee, Faculty, Administrative, Technical	*pid* → all other attributes of the relation	Yes (*pid* is the key)
Academic, NonAcademic	none (single-attribute relations)	BCNF trivially
Advises, Attached, Requires, Reserves, FundsPrj, FundsLab	none (all-key relations)	BCNF trivially
Laboratory	*labId* → *name*, *building*, *roomNumber*, *discipline*, *supervisorPid*	Yes (*labId*)
ResearchProject	*code* → *title*, *startDate*, *endDate*, *status*	Yes (*code*)
Participates	*pid*, *projectCode* → *role*	Yes (composite key)
Budget	*budgetLine* → *amountGranted*, *amountDisbursed*, *startDate*, *endDate*, *managerPid*	Yes (*budgetLine*)
EquipmentModel	*modelId* → *commercialName*, *manufacturer*, *category*, *requiredEnvironment*, *trainingMandatory*	Yes (*modelId*)
EquipmentUnit	*serialNo* → *acquisitionDate*, *purchaseCost*, *status*, *portable*, *modelId*, *labId*	Yes (*serialNo*)
Certification	*code* → *title*, *issuingAuthority*, *validityPeriod*, *safetyLevel*	Yes (*code*)
Holds	*pid*, *certCode* → *issueDate*, *expirationDate*, *grade*	Yes (composite key)
Reservation	*resId* → *submissionTS*, *plannedStart*, *plannedEnd*, *purpose*, *status*, *madeByPid*, *projectCode*, *approverPid*	Yes (*resId*)
Maintenance	*serialNo*, *startTS* → *endTS*, *type*, *description*, *cost*, *outcome*, *technicalPid*	Yes (composite key)
CalibrationRecord	*serialNo*, *calibDate* → *calibrationType*, *result*, *nextDueDate*, *remarks*	Yes (composite key)
Consumable	*consId* → *name*, *unitOfMeasure*, *hazardLevel*, *reorderThreshold*	Yes (*consId*)
Supplier	*suppId* → *name*, *contactEmail*, *phone*	Yes (*suppId*)
Supplies	*suppId*, *consId*, *labId* → *unitPrice*	Yes (composite key)
Stocks	*labId*, *consId* → *quantityOnHand*, *lastRestockDate*, *storageCondition*, *technicalPid*, *monitoringSince*	Yes (composite key)
Consumes	*resId*, *labId*, *consId* → *quantityUsed*	Yes (composite key)

2 Normalization of the Hypothetical Relations

2.1 Equipment Inventory

EquipmentInventory(*serialNo*, *modelId*, *manufacturer*, *labId*)

- **FDs.** $serialNo \to \{modelId, labId\}$ and $modelId \to manufacturer$.
- **Candidate key.** *serialNo*, since $serialNo^+$ contains all four attributes.
- **BCNF?** No. $modelId \to manufacturer$ violates BCNF because *modelId* is not a superkey ($modelId^+ = \{modelId, manufacturer\}$ does not contain *serialNo*). It is also a transitive dependency $serialNo \to modelId \to manufacturer$ on a non-prime attribute, so 3NF fails too.
- **Strongest normal form.** 1NF.
- **Decomposition.** $R_1(modelId, manufacturer)$ with $modelId \to manufacturer$ (BCNF ✓); $R_2(serialNo, modelId, labId)$ with $serialNo \to \{modelId, labId\}$ (BCNF ✓).
- **Lossless join.** $R_1 \cap R_2 = \{modelId\}$ and $modelId \to R_1$, so the decomposition is **lossless**.
- **Dependency preservation.** Preserved: each FD lies inside one projection, so $(F_1 \cup F_2)^+ = F^+$.

2.2 Project Participation

ProjectParticipation(*pid*, *projectCode*, *role*, *piPid*)

- **FDs.** $(pid, projectCode) \to role$ and $projectCode \to piPid$.
- **Candidate key.** $(pid, projectCode)$.
- **BCNF?** No. $projectCode \to piPid$ violates BCNF because *projectCode* is only part of the key and not a superkey ($projectCode^+ = \{projectCode, piPid\}$).
- **Strongest normal form.** 1NF only: *piPid* depends on part of the key, which is a partial dependency.
- **Decomposition.** $R_1(projectCode, piPid)$ with $projectCode \to piPid$ (BCNF ✓); $R_2(pid, projectCode, role)$ with $(pid, projectCode) \to role$ (BCNF ✓).
- **Lossless join.** $R_1 \cap R_2 = \{projectCode\}$ and $projectCode \to R_1$, so the decomposition is **lossless**.
- **Dependency preservation.** Preserved: $(F_1 \cup F_2)^+ = F^+$.

2.3 Equipment Certification Requirement

EquipmentCertification(*modelId*, *certCode*, *certTitle*, *issuingAuthority*)

- **FDs.** $certCode \to \{certTitle, issuingAuthority\}$. A model may require several certifications and a certification may be required by several models, so the pair $(modelId, certCode)$ identifies a requirement.
- **Candidate key.** $(modelId, certCode)$.
- **BCNF?** No. $certCode \to \{certTitle, issuingAuthority\}$ violates BCNF because *certCode* is not a superkey (it does not determine *modelId*).
- **Strongest normal form.** 1NF only: *certTitle* and *issuingAuthority* depend on part of the key (partial dependency).
- **Decomposition.** $R_1(certCode, certTitle, issuingAuthority)$ holds $certCode \to \{certTitle, issuingAuthority\}$ (BCNF ✓); $R_2(modelId, certCode)$, an all-key relation (BCNF ✓).
- **Lossless join.** $R_1 \cap R_2 = \{certCode\}$ and $certCode \to R_1$, so the decomposition is **lossless**.
- **Dependency preservation.** Preserved: the only non-trivial FD lies inside R_1.

2.4 Supplier Offer

SupplierOffer(*suppId*, *consId*, *labId*, *supplierName*, *unitPrice*)

- **FDs.** $suppId \to supplierName$ and $(suppId, consId, labId) \to unitPrice$.
- **Candidate key.** $(suppId, consId, labId)$.
- **BCNF?** No. $suppId \to supplierName$ violates BCNF because *suppId* is not a superkey.

- **Strongest normal form.** 1NF only: *supplierName* depends on part of the key (partial dependency).
- **Decomposition.** $R_1(suppId, supplierName)$ with $suppId \to supplierName$ (BCNF ✓); $R_2(suppId, consId, labId, unitPrice)$ with $(suppId, consId, labId) \to unitPrice$ (BCNF ✓).
- **Lossless join.** $R_1 \cap R_2 = \{suppId\}$ and $suppId \to R_1$, so the decomposition is **lossless**.
- **Dependency preservation.** Preserved: $(F_1 \cup F_2)^+ = F^+$.

2.5 Reservation and Project Information

ReservationProject$(resId, projectCode, projectTitle, madeByPid, status)$

- **FDs.** $resId \to \{projectCode, madeByPid, status\}$ and $projectCode \to projectTitle$.
- **Candidate key.** $resId$, since $resId^+$ contains all five attributes.
- **BCNF?** No. $projectCode \to projectTitle$ violates BCNF because $projectCode$ is not a superkey ($projectCode^+ = \{projectCode, projectTitle\}$ does not contain $resId$). Together with $resId \to projectCode$ it forms the transitive dependency $resId \to projectCode \to projectTitle$ on a non-prime attribute, so 3NF fails too.
- **Strongest normal form.** 1NF.
- **Decomposition.** $R_1(projectCode, projectTitle)$ with $projectCode \to projectTitle$ (BCNF ✓); $R_2(resId, projectCode, madeByPid, status)$ with $resId \to \{projectCode, madeByPid, status\}$ (BCNF ✓). In R_2, $projectCode$ is a foreign key referencing R_1.
- **Lossless join.** $R_1 \cap R_2 = \{projectCode\}$ and $projectCode \to R_1$, so the decomposition is **lossless**.
- **Dependency preservation.** Preserved: $projectCode \to projectTitle$ is checked in R_1 and $resId \to \{projectCode, madeByPid, status\}$ in R_2, so $(F_1 \cup F_2)^+ = F^+$ and no join is needed.

3 Team Report

3.1 Role of each team member

The whole assignment was carried out as a group, without a fixed split of the work. Every member (Malak Nhami, Marwa Mazouz, Mohamed Moussaid and Sara Lamnaiy) analyzed the functional dependencies from Lab 4 and evaluated BCNF compliance on their own. We then met to compare our decompositions for Question 2, check the lossless-join property and confirm dependency preservation. All members contributed equally to writing, revising and verifying this report.

3.2 Key challenges and how they were resolved

- **Telling 3NF and BCNF violations apart.** In `ReservationProject` the dependency *resId* $\rightarrow$ *projectCode* $\rightarrow$ *projectTitle* is transitive, so *projectCode* had to be checked separately as a determinant. We resolved it by checking, for every left-hand side, whether it is a superkey using attribute closures.
- **Lossless-join checks.** We verified that $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$ for every decomposition, i.e. that the common attributes form a key of at least one projection.
- **Dependency preservation.** For each decomposition we checked that every original FD lies entirely inside one projection, so no join is needed to enforce it during updates.
- **Finding the strongest normal form.** For each relation we distinguished partial dependencies (2NF violation) from transitive ones (3NF violation) before applying the BCNF test.

3.3 Justification of the conversion choices

Decomposing the denormalized relations into BCNF projections removes update, insertion and deletion anomalies. For example, separating (*modelId*, *manufacturer*) from `EquipmentInventory` allows a new equipment model to be recorded without an existing physical unit, and isolating *suppId* $\rightarrow$ *supplierName* avoids repeating the supplier name in every price offer. Similarly, moving (*projectCode*, *projectTitle*) out of `ReservationProject` stores each title once instead of once per reservation.

3.4 Assumptions

- The FDs given in the assignment are the complete set of business constraints for each relation.
- Keys such as *serialNo*, *resId*, *projectCode* and *suppId* are unique across all laboratories.
- An equipment model may require several certifications and a certification may be required by several models, so (*modelId*, *certCode*) is the key of `EquipmentCertification`.